

Reviewer Pack — Minimal Diophantine Exponent and SAT Clause Depth Correlatio...

Entry c1b339c91ac6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 02:12:22 UTC

Minimal Diophantine Exponent and SAT Clause Depth Correlation

Entry ID: c1b339c91ac6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 02:12:22 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Equations) **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

For every satisfiable Boolean formula φ with n variables, the minimal diophantine exponent required to express the truth table of φ is linearly correlated with the clause depth of φ , such that the minimal diophantine exponent $e(\varphi) = \Theta(\log(n)^2 * d(\varphi))$ where $d(\varphi)$ is the clause depth.

Rationale (proposer's reasoning):

Diophantine equations have been used in complexity theory to study the expressive power of computational models. By examining the minimal diophantine exponent, we can explore the inherent structure in Boolean formulas and potentially find new relationships between arithmetic complexity and satisfiability problems. The correlation with clause depth suggests that the complexity of expressing truth tables may be related to the complexity of finding satisfying assignments for the formula.

Taxonomy category: DIOPHANTINE_EQUATIONS_SAT (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 138639e042b2b2d4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between clause depth and minimal diophantine exponent is ≥ 0.8 , calculated from at least 30 seeds with a mean of $|e(\varphi) - \log(n)^2 * d(\varphi)| \leq 3$ across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal Diophantine exponent" AND "Boolean satisfiability" AND correlation - "Diophantine equations" IN BOOLEAN MODE AND "clause complexity" - diophantine exponent linearly correlated with clause depth Boolean satisfiability

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_sat_instance(n, max_depth):
        clauses = []
        for _ in range(random.randint(1, n)):
            clause_length = random.randint(1, min(max_depth, n))
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(clause_length)]
            clauses.append(clause)
        return clauses

    def truth_table_to_diophantine(clauses):
        n = len(clauses[0])
        binary = ['0'] * n
        while True:
            found_solution = True
            for clause in clauses:
                if all(binary[j-1] == '1' or c * int(binary[abs(c)-1]) >= 0 for j, c in enumerate(clause)):
                    continue
                found_solution = False
                break
            if found_solution:
                return len(binary)
            binary[-1] = '1' if binary[-1] == '0' else '0'
        for i in range(n-2, -1, -1):
            if binary[i] == '1':
                binary[i] = '0'
```

```

        binary[i+1] = '1'
        break

def calculate_correlation(trials):
    n_values = [len(trial['clauses']) for trial in trials]
    e_values = [trial['diophantine_exponent'] for trial in trials]
    d_values = [trial['clause_depth'] for trial in trials]

    mean_n = sum(n_values) / len(n_values)
    mean_e = sum(e_values) / len(e_values)
    mean_d = sum(d_values) / len(d_values)

    numerator = sum((n - mean_n) * (e - mean_e) * (d - mean_d) for n, e, d in zip(n_values, e_values, d_values))
    denominator = math.sqrt(sum((n - mean_n)**2 * (e - mean_e)**2 * (d - mean_d)**2 for n, e, d in zip(n_values, e_values, d_values)))

    return numerator / denominator if denominator != 0 else 0

trials = []
for _ in range(30):
    n = random.choice([5, 10, 15, 20, 30, 40])
    max_depth = random.randint(1, n)
    clauses = generate_random_sat_instance(n, max_depth)
    diophantine_exponent = truth_table_to_diophantine(clauses)
    trials.append({
        'n': n,
        'clauses': clauses,
        'diophantine_exponent': diophantine_exponent,
        'clause_depth': max_depth
    })

correlation_coefficient = calculate_correlation(trials)
mean_error = sum(abs(e - math.log(n)**2 * d) for n, e, d in zip([t['n'] for t in trials], [t['diophantine_exponent'] for t in trials], [t['clause_depth'] for t in trials]))

return {
    "metric_name": "Pearson Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": 30,
    "n_max": max([t['n'] for t in trials]),
    "conjecture_holds": correlation_coefficient >= 0.8 and mean_error <= 3,
    "counterexample": "" if correlation_coefficient >= 0.8 else f"Correlation coefficient: {correlation_coefficient}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(r['metric_value'] for r in results) / len(results)
    std_value = math.sqrt(sum((r['metric_value'] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

```

```

if all(r['conjecture_holds'] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
elif any(not r['conjecture_holds'] for r in results) and support_fraction >= 0.8:
    print(f"RESULT: FALSIFIED counterexample=\"Correlation coefficient below threshold\" first
else:
    print("RESULT: INCONCLUSIVE support_fraction_below_threshold")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c185f0c1.py", line 93, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c185f0c1.py", line 67, in run_trial
    diophantine_exponent = truth_table_to_diophantine(clauses)
                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c185f0c1.py", line 35, in truth_table_t
    if all(binary[j-1] == '1' or c * int(binary[abs(c)-1]) >= 0 for j, c in enumerate(clause)):
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c185f0c1.py", line 35, in <genexpr>
    if all(binary[j-1] == '1' or c * int(binary[abs(c)-1]) >= 0 for j, c in enumerate(clause)):
    ~~~~~^~~~~~
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to calculate the Pearson correlation coefficient and mean values required to support or falsify the conjecture. | next: Re-run the test with proper error handling to ensure that it completes execution and produces the necessary data for calculating the Pearson correlation coefficient and mean values.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14496
2	preregistration	ollama_remote	glm4:latest	0	0	9898

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	16699
4	novelty	ollama_remote	glm4:latest	0	0	9613
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	51296
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12213
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11906
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14031
9	judge	ollama_remote	glm4:latest	0	0	19673

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 159824 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/c1b339c91ac6.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/c1b339c91ac6.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/c1b339c91ac6.tar.gz* (if generated)